The Journal of Open Source Software

mlnlib: A portable and memory-efficient multilayer network toolkit

Eszter Bokányi[1,2¶], **Rachel G. de Jong**[1], **Yuliia Kazmina**[2], **and Frank W. Takes**[1]

1 PLANET-NL, Leiden University, The Netherlands **2** PLANET-NL, University of Amsterdam, The Netherlands ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Richard Littauer

Submitted: 19 October 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Recently, it has become possible to study the social network structure of entire societies by deriving social connections from central administrative registers of countries (Bokányi et al., 2023; Cremers et al., 2025; Panayiotou et al., 2025). These networks are often large in size, with tens of millions of nodes and billions of edges. Moreover, edges can be of multiple different types, represented as network layers (Boccaletti et al., 2014; Kivelä et al., 2014). The size and multilayer nature of these data pose challenges for efficient storage and manipulation, especially in constrained research environments where these privacy-sensitive datasets are usually available for academic research.

With `mlnlib`, we provide a Python package for memory-efficient and lightweight storage and manipulation of large multilayer networks. The package enables efficient layer-aware filtering and conversion operations to "slice and dice" the data, while keeping the user-facing workflow simple and compatible with standard Python data science tools. The package provides the `MultiLayerNetwork` class for loading, querying, aggregating, exporting, and converting multilayer networks, and the `RawCSVtoMLN` class for preparing an intermediate file format for fast loading from raw CSV sources.

Statement of need

One commonly used strategy for working with multilayer networks is to store relation types as a generic edge attribute and repeatedly filter edge tables during analysis. However, this approach is often slow for large data and becomes impractical in constrained research environments where users cannot rely on specialized hardware or software infrastructure (for example, cluster schedulers, high-performance databases, custom compilers, or distributed systems).

Storing multilayer edges in a large supra-adjacency matrix is another option, but it can be memory-inefficient when there is a large number of node pairs with an edge in multiple layers between them.

Most graph libraries are not designed to handle large sets of tabular node attributes alongside multilayer edge data. However, these are common in population-scale social networks where the registers also allow including demographic attributes of people (e.g., information on age, gender, education, or income).

`mlnlib` addresses this gap by providing an in-memory representation that is portable, i.e., it runs as a pure Python package on any machine with a standard scientific Python stack, and practical in standardized computing environments. The design goals are:

1. Efficiently store multilayer edges encoded into a single sparse adjacency matrix.

DRAFT

2. Enable fast querying by layer combinations through bitwise operations on encoded edge values.
3. Support node-based querying using tabular node attributes handled through familiar dataframe workflows.
4. Interoperate with downstream graph analysis libraries.
5. Minimize software or hardware dependencies to ensure broad accessibility and ease of installation.

This scope is intentionally narrow: `mlnlib` does not aim to replace comprehensive multilayer graph analytics libraries. Instead, it provides an efficient intermediate representation and data-engineering layer before handoff to packages such as NetworkX and igraph (Csardi & Nepusz, 2006; Hagberg et al., 2008). Moreover, it can be easily installed as a standard Python package in managed computing environments, and it does not require specialized infrastructure for installation or execution.

State of the field

The multilayer network engineering ecosystem faces significant maturity and scalability challenges (Panayiotou et al., 2024). Current multilayer network libraries vary widely in which common operations they support (e.g., input/output formats, layer manipulation, and attribute handling), and many struggle with memory efficiency and scalability when handling large, sparse networks with contemporary data volumes.

General-purpose graph libraries such as NetworkX (Hagberg et al., 2008) and igraph (Csardi & Nepusz, 2006) are widely used and provide rich algorithmic toolkits. However, multilayer handling and tabular storage of node attributes are not the first priority of these packages: they lack explicit multilayer data structures and tabular data integration for node properties. As such, related operations are either not implemented or have to be added as generic edge and node attributes, which is inefficient at scale.

The library best representing theoretical multilayer network concepts (Kivelä et al., 2014) is `pymnet` (Nurmi et al., 2024), which provides a comprehensive set of tools for multilayer network analysis. Edges are stored in nested Python dictionaries (one per intra-layer network for multiplex networks), which is inefficient if the same edge is present in multiple layers. While these dictionaries only store existing edges, no sparse matrix format is used, and the supra-adjacency matrix can only be requested as a full dense matrix of size $NL \times NL$, where N is the number of nodes and L is the number of layers. Moreover, it does not support tabular node attributes, which are common in population-scale social networks.

`multinet` (also known as `uunet`) (Magnani et al., 2021) is an R package whose Python API is also available on PyPI, and which provides a rich toolkit for standard multilayer network metrics and analysis operations. It supports multilayer workflows including community detection and centrality measures. However, like `pymnet`, `multinet` is primarily oriented toward comprehensive analysis rather than efficient storage and preprocessing. Loading large multilayer networks can be slow, edges are stored as individual objects with several indexes per layer, and node attributes are stored as per-attribute key–value maps rather than tabular dataframes (Panayiotou et al., 2024). On the other hand, this object-based design makes it easy to add or remove individual vertices, edges, and layers after the network has been built.

`py3plex` (Škrlj et al., 2019) is a lightweight Python library focused on visualization and analysis of multilayer networks. It provides tools for intra- and inter-layer visualization, and supports common multilayer operations including aggregation, slicing, indexing, and traversal. The library also incorporates node embedding methods, which are used, among others, to accelerate layout computation. While useful for visualization and exploratory analysis, `py3plex` is not specifically optimized for memory-efficient storage of very large sparse networks or integration with tabular node attributes in data-preprocessing workflows.

DRAFT

MuxViz (De Domenico et al., 2015) is an interactive visualization and analysis tool designed to facilitate understanding of multilayer network structure and dynamics. It combines network analysis algorithms with advanced visualization techniques to explore both structural properties and dynamical processes on multilayer networks. However, MuxViz is primarily designed as an interactive analysis tool rather than as a data engineering solution; it is better suited for exploratory analysis and understanding network structure at a small scale.

Threadle (Nordlund, 2025) is a high-performance multilayer and multimode network data management system designed explicitly for population-scale administrative registers. Built in C#/.NET, it addresses similar data-engineering challenges as mlnlib, including memory-efficient storage of multilayer edges, built-in node attribute management, and support for both 1-mode and 2-mode (bipartite) relations. Some design features of Threadle draw on insights from our mlnlib implementation. However, Threadle primarily targets the R ecosystem through its threadleR interface.

While existing Python multilayer network libraries focus on analysis, visualization, and algorithmic exploration, they are not designed to efficiently handle the data-engineering challenges posed by population-scale networks with billions of edges, multiple layers, and extensive tabular node attributes, particularly in constrained computing environments where specialized infrastructure is unavailable. Instead, mlnlib provides a lightweight, portable intermediate representation that combines sparse matrix encoding of multilayer edges with familiar dataframe-based node attribute handling. This design prioritizes efficient data preparation and preprocessing over built-in availability of many graph algorithms, creating a practical bridge between raw data and established graph analysis libraries, especially useful in resource-constrained research environments.

Software design

mlnlib expects as input one or more edge list CSV files, where each row contains a source node, a target node, and the layer (relation type) of the edge. Optionally, users can also provide one or more node list CSV files with node attributes, and a layer file describing the layers; if these are missing, the node set and the layers are inferred from the edge lists. Column names are mapped to the expected ones through a configuration file, which also specifies which layers should be symmetrized (i.e., treated as undirected). From these inputs, RawCSVtoMLN builds a node file (nodes.csv.gz), an edge file (edges.npz), and a layer file (layers.csv), and stores them in a single folder. This preprocessing step only needs to be run once; afterwards, the network can be loaded quickly from these files, allowing a reproducible setup in data pipelines.

Once loaded, the core MultiLayerNetwork object stores three components:

1. Node table: a standard polars or pandas dataframe with a label (unique string identifier from the input data) and an integer id (unique integer identifier from 0 to $N-1$, where N is the number of nodes) for each node, plus optional node attributes as additional columns.
2. Edge matrix: an $N \times N$ scipy.sparse.csr_matrix, where each nonzero integer entry encodes the set of layers that the corresponding edge is present in, using powers of two.
3. Layer table: a pandas dataframe with metadata for each layer, including layer (unique integer identifier), label (unique string identifier), and binary (the unique power-of-two code assigned to that layer by mlnlib). An optional group column can associate detailed layers with an aggregate category, for example, sibling, parent, and grandparent ties grouped under family.

If layer l is assigned code 2^l, then an edge present on multiple layers stores the sum of those codes. For example, value $x = 7$ encodes layers 0, 1, and 2 because $7 = 1 + 2 + 4$. Whether an edge with stored value x is present on layer k is tested by bit masking, i.e., $(x \& 2^k) = 2^k$. Since edge values are stored as unsigned 64-bit integers, a single MultiLayerNetwork can

DRAFT

hold at most 64 distinct layers, which covers the number of relation types found in typical register-based networks.

This representation supports fast extraction of layer-specific adjacency matrices, combined-layer filtering without scanning string-valued edge attributes, and easy conversion to binary, weighted, or labeled edge list outputs. Because all layers shared by a node pair are stored in a single integer, querying an edge across any combination of layers requires only one lookup and one bitwise operation, rather than one lookup per layer. The class provides exports to `igraph`, `networkx`, and GraphML, enabling downstream analysis with established graph libraries.

The compressed sparse row format is optimized for fast reading and slicing, not for modification: adding or removing individual edges or nodes requires rebuilding parts of the underlying arrays and is therefore costly. `mlnlib` is thus designed for the typical scientific use case in which the underlying network is constructed once and then queried and filtered many times, but does not change frequently.

Research impact statement

The package was developed in the context of the PLANET-NL (formerly POPNET) project, where population-scale register and socio-demographic data require repeated multilayer slicing and aggregation. In this context, an important requirement is reproducible execution in constrained computing environments, rather than dependence on specialized infrastructure.

By combining sparse encodings for multilayer edges with dataframe-native node attributes, `mlnlib` supports workflows that bridge tabular preprocessing and graph analysis. This is especially useful when researchers need to iterate between node attribute filters and multilayer edge selection before applying downstream methods.

This toolkit has made it possible to undertake multiple recent studies on population-scale social structure, mobility, inequality, migration attitudes, and online-versus-register-based social networks (Bel et al., 2025; Bokányi et al., 2023, 2026; Kazmina et al., 2024a, 2024b, 2025; Menyhért et al., 2025). The population-scale networks containing all family, household, neighbor, work, and school ties of the entire Netherlands between 2009 and 2021 have been made available through the Statistics Netherlands (CBS) Remote Access environment in the intermediate format directly readable by the `mlnlib` package (Bokányi et al., 2025).

AI usage disclosure

Generative AI assistance was used during software and manuscript preparation. Specifically, GitHub Copilot was used for code documentation, general code tidying, and editing/correcting manuscript text. Claude Sonnet 5.5 was used to resolve GitHub issues, and Claude Opus 5.5 was used for bug fixing, release packaging, and repository maintenance. All AI-assisted outputs were reviewed, edited, and validated by the human authors, who made the design and research decisions.

Acknowledgements

The research and software development were funded by the Dutch Research Council (NWO) through the SSHOC-NL project.

References

Bel, V. de, Bokányi, E., Hank, K., & Leopold, T. (2025). A parallel kinship universe? A replication of kolk et al. (2023) with dutch register data on kinship networks. *Demographic*

DRAFT

Research, *52*, 915–938. https://doi.org/10.2307/48822805

Boccaletti, S., Bianconi, G., Criado, R., Del Genio, C. I., Gómez-Gardeñes, J., Romance, M., Sendiña-Nadal, I., Wang, Z., & Zanin, M. (2014). The structure and dynamics of multilayer networks. *Physics Reports*, *544*(1), 1–122. https://doi.org/10.1016/j.physrep.2014.07.001

Bokányi, E., de Jong, R., Kazmina, Y., & Takes, F. W. (2025). *Population-scale networks of the netherlands (PLANET-NL)* [Data set]. DataverseNL. https://doi.org/10.34894/8575OP

Bokányi, E., Heemskerk, E. M., & Takes, F. W. (2023). The anatomy of a population-scale social network. *Scientific Reports*, *13*(1). https://doi.org/10.1038/s41598-023-36324-9

Bokányi, E., Kazmina, Y., Heemskerk, E. M., & Takes, F. W. (2026). Fragmentation of a longitudinal population-scale social network: Decreasing structural social cohesion in the netherlands. *arXiv*. https://arxiv.org/abs/2602.00234

Cremers, J., Kohler, B., Maier, B. F., Eriksen, S. N., Einsiedler, J., Christensen, F. K., Lehmann, S., Lassen, D. D., Mortensen, L. H., & Bjerre-Nielsen, A. (2025). Unveiling the social fabric through a temporal, nation-scale social network and its characteristics. *Scientific Reports*, *15*(1), 18383. https://doi.org/10.1038/s41598-025-98072-2

Csardi, G., & Nepusz, T. (2006). The igraph software package for complex network research. *InterJournal, Complex Systems*, *1695*. https://igraph.org

De Domenico, M., Porter, M. A., & Arenas, A. (2015). MuxViz: A tool for multilayer analysis and visualization of networks. *Journal of Complex Networks*, *3*(2), 159–176. https://doi.org/10.1093/comnet/cnu038

Hagberg, A., Schult, D., & Swart, P. (2008). NetworkX: Network analysis in python. *Proceedings of the 7th Python in Science Conference*, 11–15. https://doi.org/10.25080/Majora-92bf1922-00a

Kazmina, Y., Heemskerk, E. M., Bokányi, E., & Takes, F. W. (2024a). From contact to threat: A social network perspective on perceptions of immigration. *arXiv*. https://arxiv.org/abs/2407.06820

Kazmina, Y., Heemskerk, E. M., Bokányi, E., & Takes, F. W. (2024b). Socio-economic segregation in a population-scale social network. *Social Networks*, *78*, 279–291. https://doi.org/10.1016/j.socnet.2024.02.005

Kazmina, Y., Heemskerk, E. M., Kooij, E. van der, Bokányi, E., & Takes, F. W. (2025). Can social capital remedy structural inequality? Economic mobility in a longitudinal population-scale social network. *arXiv*. https://arxiv.org/abs/2508.05275

Kivelä, M., Arenas, A., Barthélemy, M., Gleeson, J. P., Moreno, Y., & Porter, M. A. (2014). Multilayer networks. *Journal of Complex Networks*, *2*(3), 203–271. https://doi.org/10.1093/comnet/cnu016

Magnani, M., Rossi, L., & Vega, D. (2021). Analysis of multiplex social networks with R. *Journal of Statistical Software*, *98*(8). https://doi.org/10.18637/jss.v098.i08

Menyhért, M., Bokányi, E., Corten, R., Heemskerk, E. M., Kazmina, Y., & Takes, F. W. (2025). Connectivity and community structure of online and register-based social networks. *EPJ Data Science*, *14*(1). https://doi.org/10.1140/epjds/s13688-025-00522-4

Nordlund, C. (2025). *Threadle: Population-scale network data management system*. Software system and documentation. https://www.threadle.dev/

Nurmi, T., Badie-Modiri, A., Coupette, C., & Kivelä, M. (2024). Pymnet: A python library for multilayer networks. *Journal of Open Source Software*, *9*(99), 6930. https://doi.org/10.21105/joss.06930

Panayiotou, G., Magnani, M., & Pinaud, B. (2024). Current challenges in multilayer network

DRAFT

engineering. *Applied Network Science*, *9*(1), 75. https://doi.org/10.1007/s41109-024-00686-4

Panayiotou, G., Wohlert, I. K., Bask, M., Bask, M., Magnani, M., & Mäkinen, I. H. (2025). Anatomy of a swedish population-scale network. *Scientific Reports*, *15*(1), 30300. https://doi.org/10.1038/s41598-025-15966-x

Škrlj, B., Kralj, J., & Lavrač, N. (2019). Py3plex toolkit for visualization and analysis of multilayer networks. *Applied Network Science*, *4*(1), 94. https://doi.org/10.1007/s41109-019-0203-7

DRAFT